DESENVOLVIMENTO MOBILE · MÓDULO 03 · COMUM ÀS TRILHAS · NÍVEL INICIANTE

Engenharia de Software e Git

Ciclo de vida, métodos ágeis, requisitos, modelagem, princípios de projeto, código limpo, controle de versão com Git e colaboração no GitHub

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar o que é engenharia de software e comparar modelos de processo.
- Organizar o trabalho com Scrum e Kanban.
- Escrever requisitos e histórias de usuário com critérios de aceitação.
- Modelar um app com diagramas UML e C4.
- Aplicar coesão, baixo acoplamento, SOLID e código limpo.
- Usar Git com confiança e colaborar no GitHub com *branches*, *pull requests* e revisão.

Roteiro

Neste módulo

- O que é engenharia de software
- Métodos ágeis
- Requisitos
- Modelagem
- Princípios de projeto e código limpo
- Controle de versão com Git
- Colaboração no GitHub
- Fechamento

O que é engenharia de software

Programar não é o mesmo que fazer software

Engenharia de software

Aplicação de uma abordagem **sistemática, disciplinada e mensurável** ao desenvolvimento, à operação e à manutenção de software (definição do IEEE).

Além do código

Entender o problema, planejar, projetar, testar, entregar, medir e manter, em equipe, com prazos, custos e qualidade.

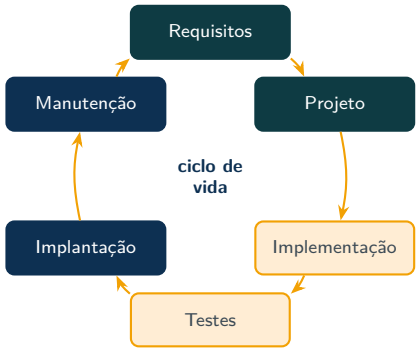
Um pouco de história

Em 1968, uma conferência da OTAN na Alemanha discutiu a "crise do software": projetos atrasados, caros e cheios de defeitos. O termo *engenharia de software* ganhou força ali, como chamado para tratar software com o rigor das engenharias.

Analogia

Um programador sozinho faz uma casinha de cachorro; para um prédio, é preciso projeto, cálculo, cronograma, equipe e fiscalização.

Atividades do ciclo de vida



Atividade	Pergunta central
Requisitos	o que o sistema deve fazer, para quem e com que qualidade?
Projeto	como organizar o sistema em partes e como elas conversam?
Implementação	como escrever o código?
Testes	funciona como deveria?
Implantação	como chega aos usuários?
Manutenção	como corrigir, adaptar e evoluir?

Atenção

A maior parte do custo de um software ao longo da vida está na **manutenção**. Código legível e testado é investimento, não luxo.

Modelos de processo

Modelo	Ideia	Ponto forte	Risco
Cascata	uma fase após a outra, cada uma concluída antes da próxima	previsível quando os requisitos são estáveis	descobrir problemas só no fim
Incremental	entregar o sistema em partes que se somam	valor entregue cedo	arquitetura precisa prever o crescimento
Espiral	ciclos guiados pela análise de riscos	bom para projetos grandes e arriscados	complexo de gerenciar
Ágil	ciclos curtos, entregas frequentes, *feedback* constante	adapta-se a mudanças	exige disciplina e cliente presente

Um pouco de história

O artigo de Winston Royce (1970), lembrado como origem da cascata, já apontava os riscos de testar só no final e sugeria iterações.

Dica

Apps móveis mudam rápido (sistemas, lojas, usuários): por isso o mercado mobile trabalha quase sempre de forma iterativa e ágil.

Métodos ágeis

O Manifesto Ágil

Valorizamos mais...		...do que
indivíduos e interações	>	processos e ferramentas
software em funcionamento	>	documentação abrangente
colaboração com o cliente	>	negociação de contratos
responder a mudanças	>	seguir um plano

Leitura correta

Os itens da direita têm valor; os da esquerda têm **mais** valor. Ágil não é "sem documentação" nem "sem plano".

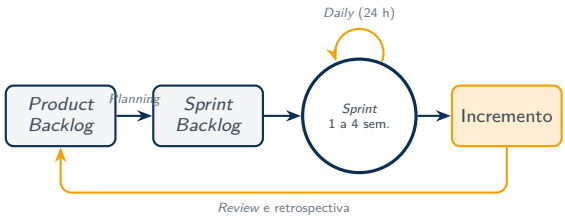
Um pouco de história

Em 2001, dezessete desenvolvedores reunidos em Utah escreveram o manifesto, com quatro valores e doze princípios.

Princípios em destaque

- Entregar software funcionando com frequência.
- Aceitar mudanças, mesmo tarde.
- Ritmo sustentável.
- Simplicidade: maximizar o trabalho **não** feito.
- Refletir e ajustar a cada ciclo.

Scrum

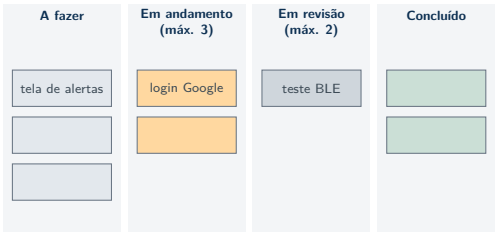


Elemento	Função
Product Owner	prioriza o *backlog*; representa o valor para o cliente
Scrum Master	remove impedimentos e cuida do processo
Developers	constroem o incremento
Daily	15 minutos para sincronizar o dia
Review	mostrar o incremento e coletar *feedback*
Retrospectiva	melhorar a forma de trabalhar

Scrum

Framework ágil leve, com papéis, eventos e artefatos fixos, que organiza o trabalho em ciclos de duração fixa chamados *sprints*.

Kanban



Kanban

Método de gestão do **fluxo** de trabalho: tarefas visíveis num quadro e **limite de trabalho em andamento** (WIP) em cada coluna.

Por que limitar o WIP

Começar menos e terminar mais: muitas tarefas abertas ao mesmo tempo atrasam todas. Mede-se o **tempo de ciclo** de cada cartão.

	Scrum	**Kanban**
Ritmo	*sprints* de duração fixa	fluxo contínuo
Papéis	PO, Scrum Master, *developers*	não exige papéis novos
Mudanças	entram na próxima *sprint*	a qualquer momento, respeitando o WIP

Requisitos

Requisitos funcionais e não funcionais

Requisito funcional

O que o sistema deve **fazer**: uma função ou comportamento.

No LabMonitor

- Listar as bancadas com o estado atual.
- Reservar uma bancada por horário.
- Notificar temperatura acima do limite.

Requisito não funcional

Como o sistema deve ser: qualidades e restrições.

Qualidade	Exemplo mensurável
Desempenho	painel abre em menos de 2 s
Disponibilidade offline	últimas leituras visíveis sem rede
Bateria	menos de 3 % por hora com BLE ativo
Acessibilidade	alvos de toque de 48 dp; leitor de tela
Segurança e LGPD	tokens cifrados; exclusão de conta
Compatibilidade	Android 8+ e iOS 15+

Atenção

Requisito vago (“o app deve ser rápido”) não pode ser testado. Escreva requisitos **verificáveis**, com números.

Histórias de usuário

História de usuário

Descrição curta de uma necessidade, do ponto de vista de quem usa:

Como *papel*, **quero** *ação*, **para** *benefício*.

História com critérios de aceitação

Como aluno, **quero** reservar uma bancada pelo celular **para** garantir meu horário de prática.

Dado que a B7 está livre das 14h às 16h, **quando** eu a reservo, **então** ela aparece como minha.
Dado que a B7 já está reservada, **quando** tento reservá-la no mesmo horário, **então** vejo "Horário indisponível".

INVEST	**Uma boa história é...**
Independente	entregável sem depender de outra
Negociável	detalhes combinados em conversa
Valiosa	gera valor para alguém
Estimável	a equipe consegue estimar
***S**mall* (pequena)	cabe numa *sprint*
Testável	tem critérios de aceitação

Definição de pronto

Lista que vale para todas as histórias: código revisado, testes passando, acessível, documentado, integrado.

Priorizar e estimar

MoSCoW	Exemplo no LabMonitor
Must (deve)	login, lista de bancadas, reserva
Should (deveria)	alertas de temperatura
Could (poderia)	modo escuro, widget na tela inicial
Won't (não agora)	versão para relógio

Valor × esforço

Comece pelo que traz muito valor com pouco esforço; questione o que traz pouco valor com muito esforço.

Pontos de história

Medida **relativa** de esforço, complexidade e incerteza (em geral na sequência 1, 2, 3, 5, 8, 13). Compara-se uma história com outra já conhecida, em vez de estimar horas.

Planning poker

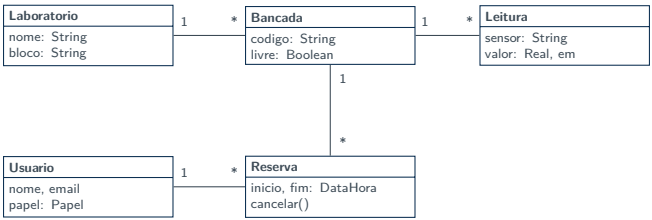
Cada pessoa mostra sua estimativa ao mesmo tempo; as divergências viram conversa, que revela riscos e mal-entendidos.

Dica

Velocidade: pontos concluídos por *sprint*. Serve para planejar a equipe, nunca para comparar pessoas.

Modelagem

UML: diagrama de classes do domínio



UML

Unified Modeling Language: notação padrão para desenhar a estrutura e o comportamento de sistemas.

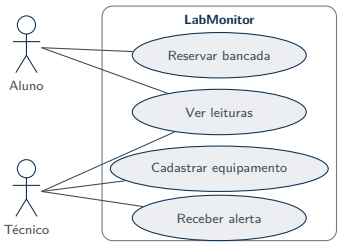
Diagrama de classes

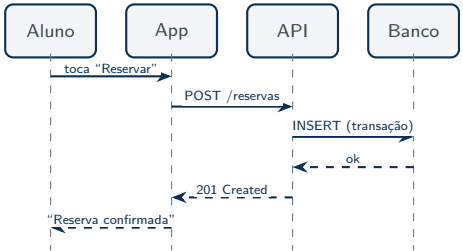
Mostra as entidades, seus atributos e operações e as **associações** entre elas com multiplicidade (1, ∗).

Dica

Esse diagrama vira as tabelas do Módulo 09 e os modelos de domínio do Módulo 08.

UML: casos de uso e sequência

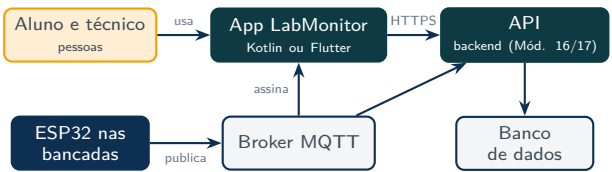




Dois olhares

Casos de uso: quem usa o sistema e para quê. **Sequência**: a ordem das mensagens entre as partes num cenário.

Modelo C4: arquitetura em níveis de zoom



Modelo C4

Quatro níveis de diagramas: **Contexto** (o sistema e quem o usa), **Contêineres** (apps, APIs, bancos), **Componentes** (partes de cada contêiner) e **Código**.

Dica

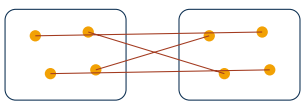
O diagrama ao lado mistura contexto e contêineres: o suficiente para qualquer pessoa nova no projeto entender o todo em um minuto.

Princípios de projeto e código limpo

Coesão e acoplamento



alta coesão, baixo acoplamento



baixa coesão, alto acoplamento

Coesão

Quanto os elementos de um módulo trabalham para um **mesmo objetivo**. Queremos alta.

Acoplamento

Quanto um módulo depende dos detalhes de outro. Queremos baixo: mudar um não deve quebrar o outro.

Analogia

Uma boa cozinha: cada gaveta guarda coisas do mesmo tipo (coesão), e trocar o fogão não exige refazer a pia (acoplamento baixo).

SOLID

Princípio	Significado	No LabMonitor
S — Responsabilidade única	uma classe tem um só motivo para mudar	a tela não faz HTTP; o serviço não desenha
O — Aberto/fechado	aberto para extensão, fechado para modificação	nova fonte de leituras (MQTT) sem alterar o painel
L — Substituição de Liskov	subtipos podem substituir o tipo base sem surpresas	`FonteFake` funciona onde `FonteBle` funciona
I — Segregação de interfaces	interfaces pequenas e específicas	`LeitorDeSensor` separado de `ConfiguradorDePlaca`
D — Inversão de dependência	depender de abstrações, não de implementações	o view model recebe `BancadaRepository`, não a classe concreta

Um pouco de história

Os princípios foram reunidos por Robert C. Martin no início dos anos 2000; o acrônimo foi sugerido por Michael Feathers.

Atenção

Princípios são guias, não leis. Aplicá-los além da conta cria camadas e interfaces que ninguém precisa.

Inversão de dependência nas duas trilhas

Kotlin

```kotlin
interface BancadaRepository {
  suspend fun listar(): List<Bancada>
}
class BancadaRepositoryRemoto(
  private val api: ApiService
) : BancadaRepository {
  override suspend fun listar() =
    api.bancadas().map { it.paraModelo() }
}
class PainelViewModel(
  private val repo: BancadaRepository  // abstração
) : ViewModel()
```

Dart

```dart
abstract interface class BancadaRepository {
  Future<List<Bancada>> listar();
}
class BancadaRepositoryRemoto
    implements BancadaRepository {
  BancadaRepositoryRemoto(this.api);
  final ApiService api;
  @override
  Future<List<Bancada>> listar() async =>
      [for (final d in await api.bancadas())
      d.paraModelo()];
}
class PainelViewModel extends ChangeNotifier {
  PainelViewModel(this.repo);   // abstração
  final BancadaRepository repo;
}
```

O view model não sabe se os dados vêm da API, do banco local ou de um *fake* de teste: só conhece o contrato.

Código limpo

Práticas

- **Nomes que revelam a intenção**: `reservasDoDia`, não `lista2`.
- **Funções pequenas**, que fazem uma coisa.
- **Sem números mágicos**: `limiteAlertaCelsius = 30`.
- Comentários explicam o **porquê**; o código mostra o **como**.
- Tratar erros perto de onde acontecem.

DRY, KISS, YAGNI

Não se repita; mantenha simples; não construa o que ainda não é necessário.

Mau cheiro	Sinal de que...
Método longo	faz coisas demais
Classe deus	concentra responsabilidades de várias
Código duplicado	falta uma abstração
Lista longa de parâmetros	falta um objeto
Comentário explicando o óbvio	o nome não está bom

Refatoração

Mudar a estrutura interna do código **sem mudar o comportamento**, em pequenos passos protegidos por testes.

Padrões de projeto

Padrão de projeto

Solução reutilizável e nomeada para um problema recorrente de projeto. Dá à equipe um **vocabulário** comum.

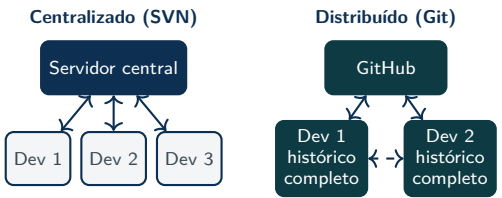
Um pouco de história

O livro *Design Patterns* (1994), de Gamma, Helm, Johnson e Vlissides (a "Gangue dos Quatro"), catalogou 23 padrões em três categorias.

Padrão	Onde aparece no curso
Factory (criacional)	criar o cliente HTTP configurado
Builder (criacional)	montar requisições e notificações
Adapter (estrutural)	DTO da API → modelo de domínio
Observer (comportamental)	*streams*, *flows* e *listeners*
Strategy (comportamental)	fonte de leituras: BLE, MQTT ou *fake*
Command (comportamental)	ações do view model (Módulo 08)
Repository	acesso a dados atrás de uma interface

Controle de versão com Git

Por que controlar versões



O que o controle de versão dá

Histórico de cada mudança (quem, quando, por quê), volta a qualquer versão, trabalho em paralelo sem sobrescrever o colega e base para revisão e CI.

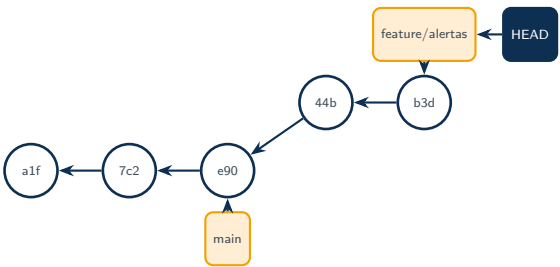
Um pouco de história

Linus Torvalds criou o Git em 2005 para o desenvolvimento do kernel Linux, priorizando velocidade, integridade e trabalho distribuído.

Dica

No Git, cada clone é um repositório completo: dá para fazer *commits* offline.

Git por dentro: *snapshots*, *hashes* e ponteiros



Cada *commit* aponta para o anterior (seu pai).

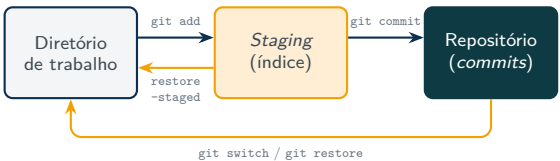
Como o Git guarda tudo

Cada *commit* é uma **foto completa** do projeto (arquivos repetidos são reaproveitados), identificada por um *hash* calculado do conteúdo. Mudou um byte, muda o *hash*: o histórico não pode ser alterado sem que se perceba.

Branch e HEAD

Uma *branch* é só um **ponteiro móvel** para um *commit*. `HEAD` indica onde você está agora.

As três áreas do Git



Para que serve o *staging*

Escolher exatamente o que entra no próximo *commit*. Um *commit* deve contar **uma** mudança coerente, mesmo que você tenha mexido em várias coisas.

```
$ git status            # o que mudou?
$ git diff              # diferenças não preparadas
$ git add lib/painel/ # preparar só isso
$ git commit -m "feat: filtro no painel"
```

Comandos essenciais

Configurar e começar

```
$ git config --global user.name "Ana Souza"
$ git config --global user.email ana@ifma.edu.br
$ git config --global init.defaultBranch main
$ git clone https://github.com/turma/labmonitor.git
$ git log --oneline --graph --all
```

Trabalhar numa branch

```
$ git switch -c feature/alertas
# ... editar, testar ...
$ git add .
$ git commit -m "feat: alerta de temperatura"
$ git push -u origin feature/alertas
```

Sincronizar e integrar

```
$ git switch main
$ git pull                # fetch + merge
$ git merge feature/alertas
$ git branch -d feature/alertas
```

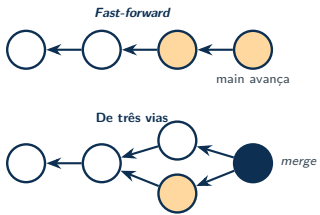
.gitignore (trechos)

```
build/
.dart_tool/           # Flutter
.gradle/              # Android
local.properties
key.properties
*.jks
.env
```

Atenção

Segredos que entraram num *commit* continuam no histórico: troque a chave.

Integrando: *merge* e conflitos



Conflito: os dois mudaram a mesma linha

```
<<<<<<< HEAD
const limiteAlerta = 30.0;
=======
const limiteAlerta = 28.5;
>>>>>>> feature/alertas
```

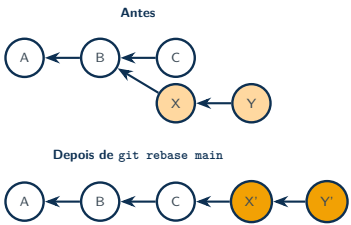
Resolver

Edite o arquivo deixando a versão correta (conversando com o colega, se preciso), remova os marcadores, rode os testes, `git add` e `git commit`.

Dica

Branches curtas e integradas com frequência geram menos conflitos.

Rebase e a regra de ouro



Rebase

Reaplica os seus *commits* sobre outra base, criando *commits* novos (X', Y'). O histórico fica linear, como se você tivesse começado depois.

Regra de ouro

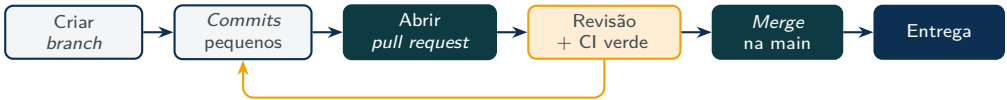
Nunca faça *rebase* de *commits* que outras pessoas já baixaram: os *hashes* mudam e o histórico delas diverge do seu.

Dica

Merge preserva a história real; *rebase* deixa a história legível. Use *rebase* só na sua *branch* local.

Colaboração no GitHub

GitHub flow



Pull request (PR)

Pedido para integrar uma *branch* em outra. Reúne descrição, diferenças, comentários de revisão e o resultado do CI num só lugar.

Proteção da *main*

Configure o repositório para exigir PR, ao menos uma aprovação e CI verde antes do *merge*. Ninguém envia direto para a *main*.

Revisão de código

O que olhar

1. Resolve o problema descrito?
2. Há testes? Eles falhariam sem a mudança?
3. É legível? Nomes, tamanho das funções, duplicação.
4. Respeita a arquitetura (camadas, dependências)?
5. Segurança e privacidade: segredos, validação, dados pessoais.
6. Acessibilidade e textos para o usuário.

Para quem envia

PRs pequenos (até cerca de 400 linhas) são revisados melhor e mais rápido. Descreva o porquê e mostre capturas de tela das mudanças visuais.

Para quem revisa

Comente o código, não a pessoa. Faça perguntas ("e se a lista vier vazia?") e diferencie o que bloqueia do que é sugestão.

Analogia

Como a revisão de um artigo científico: outro par de olhos acha o que o autor não vê.

Mensagens de *commit* e versões

`git log -oneline`

```
b3d feat(alertas): notifica acima do limite
44b fix(ble): reconecta após queda do sinal
e90 test(painel): cobre o estado de erro
7c2 docs: instruções de execução no README
a1f refactor!: repositório retorna Result
```

Conventional Commits

Padrão `tipo(escopo): descrição`. Tipos comuns: `feat`, `fix`, `docs`, `test`, `refactor`, `chore`. O `!` indica quebra de compatibilidade.

Ligação com o SemVer

`fix` → PATCH; `feat` → MINOR; quebra (`!`) → MAJOR. Ferramentas geram a versão e o *changelog* a partir das mensagens (Módulo 15).

Boa mensagem

Imperativo, curta (até cerca de 50 caracteres) e com o **porquê** no corpo quando não for óbvio.

Tag

Nome fixo para um *commit* (`v1.2.0`); no GitHub, vira uma *release* com notas e arquivos.

Issues, quadro do projeto, README e licença

Organização no GitHub

Issues registram histórias, tarefas e defeitos; **Projects** mostra as *issues* num quadro Kanban; **Milestones** agrupam o que entra numa versão. Um PR com "`Closes #12`" fecha a *issue* ao ser integrado.

README que funciona

O que é o projeto, captura de tela, como rodar, arquitetura, como contribuir e licença.

Licença	Em resumo
MIT	uso livre, inclusive comercial; manter o aviso de copyright
Apache 2.0	como a MIT, com concessão explícita de patentes
GPL-3.0	quem distribui versões modificadas deve abrir o código sob a mesma licença
Sem licença	ninguém tem permissão para reutilizar o código

Atenção

Confira a licença dos pacotes que você usa (Módulo 13: `flutter_blue_plus`) e as regras da instituição para projetos acadêmicos.

Fechamento

Resumo

Resumo

- Engenharia de software é abordagem sistemática para construir **e manter** software.
- Métodos ágeis entregam em ciclos curtos; Scrum organiza em *sprints*, Kanban limita o trabalho em andamento.
- Requisitos verificáveis, histórias INVEST e critérios de aceitação guiam o que construir.
- UML e C4 comunicam a estrutura; coesão, baixo acoplamento e SOLID guiam o projeto.
- Git guarda fotos do projeto num grafo de *commits*; *branches* são ponteiros.
- No GitHub, PR + revisão + CI protegem a *main*.

Glossário (1/2)

Termo	Significado
Ciclo de vida	sequência de atividades do software, da ideia à manutenção
Sprint	ciclo de duração fixa do Scrum que gera um incremento
WIP	trabalho em andamento; o Kanban limita sua quantidade
História de usuário	necessidade descrita como “como... quero... para...”
Critério de aceitação	condição verificável para considerar a história pronta
Requisito não funcional	qualidade ou restrição (desempenho, segurança, acessibilidade)
Coesão / acoplamento	foco interno de um módulo / dependência entre módulos
Refatoração	mudança da estrutura do código sem mudar o comportamento

Glossário (2/2)

Termo	Significado
Commit	foto do projeto com autor, data, mensagem e ponteiro para o anterior
Branch	ponteiro móvel para um *commit*; linha de trabalho independente
`HEAD`	indica o *commit* ou a *branch* atual
Staging	área onde se prepara o conteúdo do próximo *commit*
Merge / rebase	integrar históricos com um *commit* de junção / reaplicando *commits*
Pull request	pedido de integração com revisão e CI
Conventional Commits	padrão de mensagens `tipo(escopo): descrição`
Tag	nome fixo para um *commit*, usado em versões

Questões de revisão

1. Por que a manutenção domina o custo de um software, e o que isso muda na forma de programar?
2. Compare cascata e ágil quanto ao momento em que os problemas são descobertos.
3. Transforme "o app deve ser rápido" em um requisito não funcional verificável.
4. Escreva uma história de usuário com dois critérios de aceitação para "cancelar reserva".
5. Explique coesão e acoplamento com um exemplo de código ruim e sua correção.
6. Por que se diz que uma *branch* no Git é "barata"?
7. Quando usar *merge* e quando usar *rebase*? Qual é a regra de ouro?
8. Um colega enviou uma chave de API num *commit* já publicado. O que fazer?

Exercícios

Exercício 1 — Requisitos

Escreva cinco histórias de usuário do LabMonitor com critérios de aceitação e classifique-as por MoSCoW.

Exercício 2 — Modelagem

Desenhe os diagramas de classes e de sequência para "registrar uma leitura recebida por BLE".

Exercício 3 — Git em dupla

Em dupla, criem um conflito proposital na mesma linha, resolvam-no e integrem via *pull request* com revisão.

LabMonitor — Etapa 3

Crie o repositório com README, licença e `.gitignore`; registre o *backlog* como *issues* num quadro do GitHub Projects; proteja a *main* e entregue a primeira tarefa por *pull request*.

Para aprofundar

- *Pro Git* (livro gratuito, com tradução): `https://git-scm.com/book/pt-br/v2`
- Guia do Scrum: `https://scrumguides.org`
- Manifesto Ágil: `https://agilemanifesto.org/iso/ptbr/manifesto.html`
- Modelo C4: `https://c4model.com`
- *Conventional Commits*: `https://www.conventionalcommits.org/pt-br`
- Documentação do GitHub: `https://docs.github.com/pt`

Respostas das questões de revisão

1. O software vive anos sendo corrigido e adaptado (novas versões do sistema, das lojas, de requisitos). Por isso vale investir em legibilidade, testes e baixo acoplamento desde o início.
2. Na cascata, a integração e os testes vêm no fim, quando corrigir é caro; no ágil, cada ciclo curto entrega algo testável e expõe problemas cedo.
3. Exemplo: "o painel exibe a lista de bancadas em até 2 s após o toque, em um aparelho intermediário, com rede 4G".
4. "Como aluno, quero cancelar minha reserva para liberar o horário." Critérios: a reserva some da minha lista e a bancada fica livre; não posso cancelar reservas de outras pessoas.
5. Uma tela que faz HTTP, SQL e desenho tem baixa coesão e acopla a UI à API; separar em view, view model, repositório e serviço resolve.
6. Porque é só um ponteiro para um *commit*: criar ou apagar não copia arquivos.
7. *Merge* para integrar e preservar a história; *rebase* para organizar a sua *branch* local antes do PR. Nunca reescrever *commits* já compartilhados.
8. Considerar a chave comprometida: revogá-la e gerar outra imediatamente. Limpar o histórico não basta, pois cópias já podem existir.

Fim do Módulo 03

Próximo: Módulo 04 (comum) — Design de interfaces e UX: princípios, acessibilidade, Material 3 e Figma.